Spaceship needs a tune-up

Addressing some discovered issues with P0515 and P1185

Document #:	P1630R1
Date:	2019-07-17
Project:	Programming Language C++ CWG, EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Introduction

The introduction of `operator<=>` into the language ([P0515R3] with relevant extension [P0905R1]) added a novel aspect to name lookup: candidate functions can now include both candidates with different names and a reversed order of arguments. The expression `a < b` used to always only find candidates like `operator<(a, b)` and `a.operator<(b)` now also finds `(a <=> b) < 0` and `0 < (b <=>`

a `)`. This change makes it much easier to write comparisons - since you only need to write the one `operator <=>`.

However, that ended up being insufficient due to the problems pointed out in [P1190R0], and in response [P1185R2] was adopted in Kona which made the following changes:

1. Changing candidate sets for equality and inequality
   a. `<=>` is no longer a candidate for either equality or inequality
   b. `==` gains `<=>`'s ability for both reversed and rewritten candidates
2. Defaulted `==` does memberwise equality, defaulted `!=` invokes `==` instead of `<=>`.
3. Strong structural equality is defined in terms of `==` instead of `<=>`
4. Defaulted `<=>` can also implicitly declare defaulted `==`

Between P0515 and P1185, several issues have come up in the reflectors that this paper hopes to address. These issues are largely independent from each other, and will be discussed independently.

§ 2 Tomasz's example

Consider the following example [CWG2407] (note that the use of `int` is not important, simply that we have two types, one of which is implicitly convertible to the other):

```
struct A {
  operator int() const;
};

bool operator==(A, int);              // #1
// builtin bool operator==(int, int); // #2
// builtin bool operator!=(int, int); // #3

int check(A x, A y) {
  return (x == y) +  // In C++17, calls #1; in C++20, ambiguous between #1 and reversed #1
      (10 == x) +    // In C++17, calls #2; in C++20, calls #1
      (10 != x);     // In C++17, calls #3; in C++20, calls #1
}
```

There are two separate issues demonstrated in this example: code that changes which function gets called, and code that becomes ambiguous.

§ 2.1 Changing the result of overload resolution

The expression `10 == x` in C++17 had only one viable candidate: `operator==(int, int)`, converting the `A` to an `int`. But in C++20, due to P1185, equality and inequality get reversed candidates as well. Since

equality is symmetric, `10 == x` is an equivalent expression to `x == 10`, and we consider both forms. This gives us two candidates:

```
bool operator==(int, A);   // #1 (reversed)
bool operator==(int, int); // #2 (builtin)
```

The first is an Exact Match, whereas the second requires a Conversion, so the first is the best viable candidate.

Silently changing which function gets executed is facially the worst thing we can do, but in this particular situation doesn't seem that bad. We're already in a situation where, in C++17, `x == 10` and `10 == x` invoke different kinds of functions (the former invokes a user-defined function, the latter a builtin) and if those two give different answers, that seems like an inherently questionable program.

The inequality expression behaves the same way. In C++17, `10 != x` had only one viable candidate: the `operator!=(int, int)` builtin, but in C++20 also acquires the reversed and rewritten candidate `(x == 10) ? false : true`, which would be an Exact Match. Here, the status quo was that `x != 10` and `10 != x` both invoke the same function - but again, if that function gave a different answer from `!(x == 10)` or `!(10 == x)`, that seems suspect.

§ 2.2 Code that becomes ambiguous

The homogeneous comparison is more interesting. `x == y` in C++17 had only one candidate: `operator==(A, int)`, converting `y` to an `int`. But in C++20, it now has two:

```
bool operator==(A, int); // #1
bool operator==(int, A); // #1 reversed
```

The first candidate has an Exact Match in the 1st argument and a Conversion in the 2nd, the second candidate has a Conversion in the 1st argument and an Exact Match in the 2nd. While we do have a tiebreaker to choose the non-reversed candidate over the reversed candidate ([over.match.best]/2.9), that only happens when each argument's conversion sequence *is not worse than* the other candidates' ([over.match.best]/2)… and that's just not the case here. We have one better sequence and one worse sequence, each way.

As a result, this becomes ambiguous.

Note that the same thing can happen with `<=>` in a similar situation:

```
struct C {
    operator int() const;
    strong_ordering operator<=>(int) const;
};
```

```
auto f() { return C{} <=> C{}; } // error: ambiguous
```

But in this case, it's completely new code which is ambiguous - rather than existing, functional code.

§ 2.3 Similar examples

There are several other examples in this vein that are important to keep in mind, courtesy of Davis Herring.

```
struct B { B(int); };

bool operator==(B, B);

bool g() { return B() == 0; }
```

We want this example to work, regardless of whatever rule changes we pursue. One potential rule change under consideration was reversing the arguments rather than parameters, which would lead to the above becoming ambiguous between the two argument orderings.

Also:

```
struct C { operator int(); };
struct D : C {};

bool operator==(const C&, int);

bool h() { return D() == C(); }
```

The normal candidate has Conversion and User, the reversed parameter candidate has User and Exact Match, which makes this similar to Tomasz's example: valid in C++17, ambiguous in C++20 under the status quo rules.

§ 2.4 Today's Guidance

From Herb Sutter's post [herb] on the topic:

> Actually, C++20 is removing a pre-C++20 can of worms we can now unlearn.
>
> This example is "bad" code that breaks two rules we teach today, and compiling it in C++20 will make it strictly better:
>
> 1. It violates today's guidance that you should write symmetric overloads of a heterogeneous `operator==` to avoid surprises. In this case, they provided `operator==(A,int)` but failed to provide `operator==(int,A)`. As a result, today we have to explain arcane details of why `10==x` and `x==10` do different and possibly inconsistent things in this code, which forces us to explain several language rules plus teach a coding guideline to always remember to provide two overloads of a heterogeneous operator== (because if

you forget, this code will compile but do inconsistent things depending on the order of `10 ==x` and `x == 10`). That's a can of worms we can stop teaching in C++20.

2. It violates today's guidance that you should also write a homogeneous `operator ==` to avoid a performance and/or correctness pitfall. In this case, they forgot to provide `operator ==(A,A)`. As a result, today we have to explain why `x ==y` "works" in this code, but that's a bug not a feature – it "works" only because we already violated (1) above, and that it "works" is harboring a performance bug (implicit conversion) and possibly a correctness bug (if comparing two A's directly might do something different than first converting one to an int). If today the programmer had not violated (1), they would already get a compile-time error; so the fact that `x ==y` is ambiguous is not actually new in C++20, what is new is that you will now consistently always get the compile-time error that you are missing `operator ==(A,A)` instead of having it masked sometimes if you broke another rule.

So recompiling this "bad" code in C++20 mode is strictly good: For (1), C++20 silently fixes the bug, because the existing `operator ==(A, int)` now does what the user almost certainly intended. For (2), C++20 removes the pitfall by making the existing compile-time diagnostic guaranteed instead of just likely.

Operationally, the main place you'll notice a difference in C++20 is in code that a wise man once described as "code that deserves to be broken."

Educationally, C++20 does remove a pre-C++20 can of worms, but mostly the only ones who will notice are us "arcana experts" who are steeped in today's complexity (because we have to unlearn our familiar wormcan); most "normals" will only notice that now C++ does what they thought it did. :)

§ 2.5 Proposal

The model around comparisons is better in the working draft than it was in C++17. We're also now in a position where it's simply much easier to write comparisons for types - we no longer have to live in this world where everybody only declares `operator <` for their types and then everybody writes algorithms that pretend that only `<` exists. Or, more relevantly, where everybody only declares `operator ==` for their types and nobody uses `!=`. This is a Good Thing.

Coming up with a way to design the rewrite rules in a way that makes satisfies both Tomasz's and Davis's examples leads to a *very* complex set of rules, all to fix code that is fundamentally ambiguous.

This paper proposes that the status quo is the very best of the quos. Some code will fail to compile, that code can be easily fixed by adding either a homogeneous comparison operator or, if not that, doing an explicit conversion at the call sites. This lets us have the best language rules for the long future this language still has ahead of it. Instead, we add an Annex C entry.

§ 3 Cameron's Example

Cameron DaCamara submitted the following example [cameron] after MSVC implemented `operator <=>` and P1185R2:

```
template <typename Lhs, typename Rhs>
struct BinaryHelper {
  using UnderLhs = typename Lhs::Scalar;
  using UnderRhs = typename Rhs::Scalar;
  operator bool() const;
};

struct OnlyEq {
  using Scalar = int;
  template <typename Rhs>
  const BinaryHelper<OnlyEq, Rhs> operator==(const Rhs&) const;
};

template <typename...>
using void_t = void;

template <typename T>
constexpr T& declval();

template <typename, typename = void>
constexpr bool has_neq_operation = false;

template <typename T>
constexpr bool has_neq_operation<T, void_t<decltype(declval<T>() !=
declval<int>())>> = true;

static_assert(!has_neq_operation<OnlyEq>);
```

In C++17, this example compiles fine. `OnlyEq` has no `operator !=` candidate at all. But, the wording in [over.match.oper] currently states that:

> … the rewritten candidates include all member, non-member, and built-in candidates for the operator `==` for which the rewritten expression `(x == y)` is well-formed when contextually converted to `bool` using that operator `==`.

Checking to see whether `OnlyEq`'s `operator ==`'s result is contextually convertible to `bool` is not SFINAE-friendly; it is an error outside of the immediate context of the substitution. As a result, this well-formed C++17 program becomes ill-formed in C++20.

The problem here in particular is that C++20 is linking together the semantics of `==` and `!=` in a way that they were not linked before – which leads to errors in situations where there was no intent for them to have been linked.

§ 3.1 Proposal

This example we must address, and the best way to address is to carve out less space for rewrite candidates. The current rule is too broad:

> For the `!=` operator ([expr.eq]), the rewritten candidates include all member, non-member, and built-in candidates for the operator `==` for which the rewritten expression `(x == y)` is well-formed when **contextually converted to** `bool` using that operator `==`. For the equality operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each member, non-member, and built-in candidate for the operator == for which the rewritten expression `(y == x)` is well-formed when **contextually converted to** `bool` using that operator `==`.

We really don't need "contextually converted to `bool`" - in fact, we're probably not even getting any benefit as a language from taking that broad a stance. After all, if you wrote a `operator==` that returned `std::true_type`, you probably have good reasons for that and don't necessarily want an `operator!=` that returns just `bool`. And for types that are even less `bool`-like than `std::true_type`, this consideration makes even less sense.

This paper proposes that we reduce this scope to *just* those cases where `(x == y)` is a valid expression that has type exactly `bool`. This unbreaks Cameron's example – `BinaryHelper<OnlyEq, Rhs>` is definitely not `bool` and so `OnlyEq` continues to have no `!=` candidates – while also both simplifying the language rule, simplifying the specification, and not reducing the usability of the rule at all. Win, win, win.

§ 3.2 Squinty Cases

If we require `bool`, the first casualty will be the closest-to-`bool` types: `std::true_type` and `std::false_type`. Consider an example like:

```
        std::true_type operator==(EmptySequenceIterator, std::default_sentinel_t) {
return {}; }
        std::false_type operator==(InfiniteSequenceIterator, std::default_sentinel_t)
{ return {}; }
```

Do we really want to disallow `default_sentinel == EmptySequenceIterator{}` or `default_sentinel != InfiniteSequenceIterator[]`? Don't these have "obvious" meanings? Maybe. I think it's harder to say than it at first appears.

Consider `!=` first. What would the type of `default_sentinel != InfiniteSequenceIterator{}` be? `bool`, right? With the value `true`? But is that really the correct answer – wouldn't you really want it to be of type `std::true_type`? That seems more along the lines of what the user intent might be. But how would the language get there? Even these cases seem like if you want to do something special it's really up to you to do that something special.

Now let's go back to `==`. If we allow `default_sentinel == EmptySequenceIterator{}` (since `==` is… obviously symmetric right?), then what's

special about `==`? Wouldn't we also want to allow symmetry for `!=`? And `<` and `>=`? At this point, this seems like scope creep.

In any case, requiring `bool` today doesn't shut the door to any loosening of these requirements tomorrow. Let's just get the definitely-known-to-be-extremely-useful case in the door and worry about the possibly-interesting-to-consider cases later.

§ 4 Richard's Example

Daveed Vandevoorde [vdv.well-formed] pointed out that the wording in [over.match.oper] for determining rewritten candidates is currently:

> For the relational (7.6.9) operators, the rewritten candidates include all member, non-member, and built-in candidates for the operator `<=>` for which the rewritten expression `(x <=> y) @ 0` is **well-formed** using that `operator <=>`.

Well-formed is poor word choice here, as that implies that we would have to fully instantiate both the `<=>` invocation and the `@` invocation. What we really want to do is simply check if this is viable in a SFINAE-like manner. Addressing that may seem like mostly a Core matter, except that it does lead to other interesting questions of what exactly we mean by well-formed and what exactly do we want to check.

This led Richard Smith to submit the following example [smith]:

```
struct Base {
  friend bool operator<(const Base&, const Base&);  // #1
  friend bool operator==(const Base&, const Base&);
};
struct Derived : Base {
  friend std::strong_equality operator<=>(const Derived&, const Derived&); // #2
};
bool f(Derived d1, Derived d2) { return d1 < d2; }
```

The status quo is that `d1 < d2` invokes `#1`. `#2` is not actually a candidate because we have to consider the full expression `(d1 <=> d2) < 0`. While `(d1 <=> d2)` is a valid expression, `(d1 <=> d2) < 0` is not, which removes it from consideration.

The question here is: should we even consider the `@ 0` part of the rewritten expression for validity? If we did not, then `#2` would become not only a candidate but also the best candidate. As a result, `d1 < d2` becomes ill-formed by way of `#2` because `(d1 <=> d2) < 0` is not a valid expression. We're no longer hiding that issue.

The reasoning here is that by considering the `@ 0` part of the expression for determining viable candidates, we are effectively overloading on return type. That doesn't seem right.

§ 4.1 Shallowly well-formed

We cannot use the term “well-formed” in the Core language to describe what it means for, colloquially, an expression to be… “valid.” We need something shallower than that. This need comes up several times in the context of comparison operators - both for choosing reversed and rewritten candidates as well as defining defaulted ones (also pointed out by Daveed [vdv.defaulted]). This also comes up in how to word [P1186R2].

We already do something like this for the special member functions of class types. For instance, [class.default.ctor] says that:

2 A defaulted default constructor for class X is defined as deleted if:

(2.1) — […]

(2.7) — any potentially constructed subobject, except for a non-static data member with a *brace-or-equal-initializer*, has class type `M` (or array thereof) and either `M` has no default constructor or **overload resolution ([over.match]) as applied to find `M`’s corresponding constructor results in an ambiguity or in a function that is deleted or inaccessible from the defaulted default constructor**, or

(2.8) — […]

It’s that idea that we want here. Richard Smith in private correspondence suggested the term:

> Overload resolution is said to result in a *usable function* `F` if overload resolution succeeds and selects a function `F` that is not deleted and is accessible from the context in which overload resolution was performed.

§ 4.2 Proposal

This paper agrees with Richard that we should not consider the validity of the `@0` part of the comparison in determining the candidate set. In other words, overload resolution on `x < y` will look up candidates for all of `x < y`, `x <=> y`, and `y <=> x` and perform overload resolution on that full set - without considering what the return type of spaceship might be. If the best viable candidate is a spaceship candidate whose result is not an ordering, then the result is ill-formed. Likewise, overload resolution on `x != y` will look up candidates `x != y`, `y != x`, `x == y`, and `y == x` regardless of whether these return `bool`.

The provided example is well-formed in the current working draft, but becomes ill-formed as a result of this proposal. It is possible to construct examples that are valid C++17 code that become ill-formed as a result of this change:

```
struct NotBool { };

struct X {
    X(int);
    friend NotBool operator==(X, int);
```

```
        friend NotBool operator!=(X, X);
    };

    // in C++17, calls the only candidate, the operator!=(X, X).
    // As a result of this specific change, the operator==
    // is part of the candidate set and is a better match, but
    // its result type is not bool so this is ill-formed
    X() != 4;
```

I don't know if there are real-world code examples that would break.

5 Default comparisons for reference data members

The last issue, also raised by Daveed Vandevoorde ([vdv.reference]) is what should happen for the case where we try to default a comparison for a class that has data members of reference type:

```
    struct A {
        int const& r;
        auto operator<=>(A const&, A const&) = default;
    };
```

What should that do? The current wording in [class.compare.default] talks about a list of subobjects, and reference members aren't actually subobjects, so it's not clear what the intent is. There are three behaviors that such a defaulted comparison could have:

1. The comparison could be defined as deleted (following copy assignment with reference data members)
2. The comparison could compare the identity of the referent (following copy construction with reference data members)
3. The comparison could compare through the reference (following what rote expression substitution would do)

In other words:

```
    int i = 0, j = 0, k = 1;
                  // |  option 1  | option 2 | option 3 |
    A{i} == A{i}; // | ill-formed |   true   |   true   |
    A{i} == A{j}; // | ill-formed |   false  |   true   |
    A{i} == A{k}; // | ill-formed |   false  |   false  |
```

Note however that reference data members add one more quirk in conjunction with [P0732R2]: does `A` count as having strong structural equality, and what would it mean for:

```
    template <int&> struct X { };
    template <A> struct Y { };
    static int i = 0, j = 0;
    X<i> xi;
    X<j> xj;
```

```
        Y<A{i}> yi;
        Y<A{j}> yj;
```

In even C++17, `xi` and `xj` are both well-formed and have different types. Under option 1 above, the declaration of `Y` is ill-formed because `A` does not have strong structural equality because its `operator==` would be defined as deleted. Under option 2, this would be well-formed and `yi` and `yj` would have different types – consistent with `xi` and `xj`. Under option 3, `yi` and `yj` would be well-formed but somehow have the same type, which is a bad result. We would need to introduce a special rule that classes with reference data members cannot have strong structural equality.

§ 5.1 Anonymous unions

In the same post [vdv.reference], Daveed also questioned what defaulted comparisons would do in the case of anonymous unions:

```
        struct B {
            union {
                int i;
                char c;
            };
            auto operator<=>(B const&, B const&) = default;
        };
```

What does this mean? We can generalize this question to also include union-like classes - or any class that has a variant member. This is an interesting case to explore in the future, since at constexpr time such comparisons could be defined as valid whereas at normal runtime it really couldn't be. But for now, the easy answer is to consider such defaulted comparisons as being defined as deleted and to make this decision more explicit in the wording.

§ 5.2 Design intent

P0515 clearly lays out the design intent as comparison following copying, emphasis mine:

> This proposal unifies and regularizes the noncontroversial parts of previous proposals, and incorporates EWG direction to pursue three-way comparison, **letting default copying guide default comparison**, and having a simple way to write a memberwise comparison function body.

and

> For raw pointers […] I'm going with `strong_ordering`, **on the basis of maintaining a strict parallel between default copying and default comparison** (we copy raw pointer members, so we should compare them too unless there is a good reason to do otherwise […])

and

> For copyable arrays `T[N]` (i.e., that are nonstatic data members), `T[N] <=> T[N]` returns the same type as

`T`'s `<=>` and performs lexicographical elementwise comparison. For other arrays, **there is no `<=>` because the arrays are not copyable.**

5.3 Proposal

This paper proposes making more explicit that defaulted comparisons for classes that have reference data members or variant data members are defined as deleted. It's the safest rule for now and is most consistent with the design intent as laid out in P0515.

A future proposal can always relax this restriction for reference data members by pursuing option 2 above.

6 `std::pair<T,U>`, references, and strong structural equality

With the adoption of [P0732R2], C++20 will get class types as non-type template parameters… provided those class types satisfy *strong structural equality*, which now means has a defaulted `operator==` plus a few other things. One library type that would seem like it would be easy to provide strong structural equality for is `std::pair`, since its equality operation is just a memberwise equality comparison in declaration order. [P1614R1] proposes precisely that.

The relevant part of `std::pair` would become, with that proposal:

```
template <typename T, typename U>
struct pair {
    T first;
    U second;

    friend constexpr bool operator==(const pair&, const pair&) = default;
};
```

However, as Casey Carter pointed out in [carter.oops], if we take the previous proposal and define the above equality operator as deleted if either `T` or `U` is a reference type, then we would break existing code that today can compare `pairs` holding references:

```
int i = 42, j = 42;
pair<int&, int> p(i, 17);
pair<int&, int> q(j, 17);
assert(p == q); // valid code today, assertion holds
```

With concepts in C++20, we should be able to have our cake and eat it too. That is, provide a defaulted `operator==` but provide *another* non-defaulted `operator==` to handle the reference cases. Something like this:

```
template <typename T, typename U>
struct pair {
    T first;
```

```
        U second;

        friend constexpr bool operator==(const pair&, const pair&) = default;

        friend constexpr bool operator==(const pair& lhs, const pair& rhs)
            requires (is_reference_v<T> || is_reference_v<U>)
        {
            return lhs.first == rhs.first && lhs.second == rhs.second;
        }
    };
```

This ensures that we both would not break existing code *and* that we allow `pairs` to be used as non-type template parameters where the underlying types could be used as non-type template parameters.

However, our current wording for strong structural equality doesn't handle this particularly well - it just requires that a class have *an* `operator==` defined as defaulted. This happens to give the correct answer for this particular example (`pair<int&, int>` has a defaulted `operator==`, but it is defined as deleted, so it would be excluded), but it would be nicer if we got the correct answer more soundly… rather than simply by sheer happenstance.

6.1 Proposal

In [P0848R2], new wording is introduced to select what will become the destructor from potentially multiple candidates with differing constraints. The proposed wording there, courtesy of Richard Smith, is:

> At the end of the definition of a class, overload resolution is performed among the prospective destructors declared in that class with an empty argument list to select the destructor for the class. The program is ill-formed if overload resolution fails. Destructor selection does not constitute a reference to ([dcl.fct.def.delete]) or odr-use of ([basic.def.odr]) the selected destructor, and in particular, the selected destructor may be deleted.

We want similar wording for class types to select the unique best `operator==`, and then stipulate our requirements on that chosen `operator==`.

7 Wording

[Editor's note: The wording here introduces the term *usable function*, which is also introduced in P1186R2 with the same wording.]

Insert a new paragraph after 11.10.1 [class.compare.default]/1:

> A defaulted comparison operator function for class `C` is defined as deleted if any non-static data member of `C` is of reference type or `C` is a union-like class ([class.union.anon]).

Change 11.10.1 [class.compare.default]/3.2:

3 A type `C` has *strong structural equality* if, given a glvalue `x` of type `const C`, either:

(3.1) — `C` is a non-class type and `x <=> x` is a valid expression of type `std::strong_ordering` or `std::strong_equality`, or

(3.2) — `C` is a class type where all of the following hold: ~~with an `==` operator defined as defaulted in the definition of `C`, `x == x` is well-formed when contextually converted to bool, all of `C`'s base class subobjects and non-static data members have strong structural equality, and `C` has no `mutable` or `volatile` subobjects.~~

(3.2.1) — All of `C`'s base class subobjects and non-static data members have strong structural equality.

(3.2.2) — `C` has no `mutable` or `volatile` non-static data members.

(3.2.3) — At the end of the definition of `C`, overload resolution performed for the expression `x == x` succeeds and finds either a friend or public member `==` operator that is defined as defaulted in the definition of `C`.

Change 11.10.2 [class.eq]/4 to require `bool` and also more exhaustively handle the error cases:

4 A defaulted `!=` operator function for a class `C` with parameters `x` and `y` is defined as deleted if

(4.1) — overload resolution ([over.match]), as applied to `x == y` ~~(also considering synthesized candidates with reversed order of parameters ([over.match.oper])), results in an ambiguity or a function that is deleted or inaccessible from the operator function~~ does not result in a usable function, or

(4.2) — `x == y` ~~cannot be contextually converted to~~ ~~`bool`~~ is not a prvalue of type `bool`.

Otherwise, the operator function yields ~~`(x == y) ? false : true`~~ `!(x == y)`.

Change 11.10.4 [class.rel]/2 to likewise more exhaustively handle the error cases:

2 The operator function with parameters `x` and `y` is defined as deleted if

(2.1) — overload resolution ([over.match]), as applied to `x <=> y` ~~results in an ambiguity or a function that is deleted or inaccessible from the operator function~~ does not result in a usable function, or

(2.2) — operator `@` cannot be applied to the return type of `x <=> y`.

Otherwise, the operator function yields `x <=> y @ 0`.

Add to the end of 12.3 [over.match], the new term *usable function*:

Overload resolution results in a *usable function* if overload resolution succeeds and the selected function is not deleted and is accessible from the context in which overload resolution was performed.

Change 12.3.1.2 [over.match.oper]/3.4, also splitting it up into sub-bullets:

(3.4) The rewritten candidate set is determined as follows:

(3.4.1) — For the relational ([expr.rel]) operators, the rewritten candidates include all member, non-member, and built-in candidates ~~for the~~ `operator` ~~<=> for which the rewritten expression~~ ~~(x~~ `<=> y` ~~)~~ ~~@~~ ~~0 is well-formed using that operator~~ ~~<=>~~ for the expression `x <=> y`.

(3.4.2) — For the relational ([expr.rel]) and three-way comparison ([expr.spaceship]) operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each member, non-member, and built-in candidate for ~~the operator~~ ~~<=> for which the rewritten expression~~ ~~0~~ ~~@~~ ~~(y~~ ~~<=> x~~ ~~) is well-formed using that~~ `operator` ~~<=>~~ the expression `y <=> x`.

(3.4.3) — For the `!=` operator ([expr.eq]), the rewritten candidates include all member, non-member, and built-in candidates ~~for the operator == for which the rewritten expression~~ ~~(x~~ ~~== y~~ ~~) is well-formed when contextually converted to~~ `bool` ~~using that operator~~ ~~==~~ for the expression `x == y`.

(3.4.4) — For the equality operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each member, non-member, and built-in candidate ~~for the operator~~ ~~== for which the rewritten expression~~ ~~(y~~ ~~== x~~ ~~) is well-formed when contextually converted to~~ `bool` ~~using that operator~~ ~~==~~ for the expression `y == x`.

(3.4.5) — For all other operators, the rewritten candidate set is empty.

[*Note*: A candidate synthesized from a member candidate has its implicit object parameter as the second parameter, thus implicit conversions are considered for the first, but not for the second, parameter. *end note*] ~~In each case, rewritten candidates are not considered in the context of the rewritten expression. For all other operators, the rewritten candidate set is empty.~~

Split 12.3.1.2 [over.match.oper]/8 into two paragraphs, and require the type be `bool`:

8 If a rewritten `operator <=>` candidate is selected by overload resolution for ~~a relational or three-way comparison~~ an operator `@`, `x @ y` is interpreted as ~~the rewritten expression:~~ `0 @ (y <=> x)` if the selected candidate is a synthesized candidate with reversed order of parameters, or `(x <=> y) @ 0` otherwise, using the selected rewritten `operator <=>` candidate. Rewritten

candidates for the operator `@` are not considered in the context of the resulting expression.

8* If a rewritten `operator ==` candidate is selected by overload resolution for ~~a `!=` operator~~ an operator `@`, its return type shall be *cv* `bool`, and `x @ y` is interpreted as:

(8*.1) — If `@` is `!=` and the selected candidate is a synthesized candidate with reversed order of parameters, `!(y == x)`.

(8*.2) — Otherwise, if `@` is `!=`, `!(x == y)`.

(8*.3) — Otherwise, if `@` is `==`, `y == x`.

in each case using the selected rewritten `operator ==` candidate. ~~`x != y` is interpreted as `(y == x) ? false : true` if the selected candidate is a synthesized candidate with reversed order of parameters, or `(x == y) ? false : true` otherwise, using the selected rewritten `operator ==` candidate. If a rewritten candidate is selected by overload resolution for an `==` operator, `x == y` is interpreted as `(y == x) ? true : false` using the selected rewritten `operator ==` candidate.~~

Add a new entry to [diff.cpp17.over]:

Affected subclause: [over.match.oper]
Change: Equality and inequality expressions can now find reversed and rewritten candidates.
Rationale: Improve consistency of equality with three-way comparison and make it easier to write the full complement of equality operations.
Effect on original feature: Equality and inequality expressions between two objects of different types, where one is convertible to the other, could invoke a different operator. Equality and inequality expressions between two objects of the same type could become ambiguous.

```
struct A {
  operator int() const;
};

bool operator==(A, int);      // #1
// bool operator==(int, int); // #2, built-in
// bool operator!=(int, int); // #3, built-in

int check(A x, A y) {
  return (x == y) +  // ill-formed; previously well-formed
    (10 == x) +      // calls #1, previously called #2
    (10 != x);       // calls #1, previously called #3
}
```

8 Acknowledgments

Thank you very much to everyone that has diligently participated in pointing out issues with `operator <=>` and committed lots of time to email traffic with me to help produce this paper (the two groups are heavily overlapping). Thank you to Cameron DaCamara, Davis Herring, Tomasz Kamiński, Jens Maurer, Richard Smith, David Stone, Herb Sutter, and Daveed Vandevoorde.

9 References

[cameron] Cameron DaCamara. 2019. Potential issue after P1185R2 - SFINAE breaking change.
http://lists.isocpp.org/core/2019/04/5935.php

[carter.oops] Casey Carter. 2019. strong structural equality of library types.
http://lists.isocpp.org/core/2019/06/6715.php

[CWG2407] Tomasz Kamiński. 2019. Missing entry in Annex C for defaulted comparison operators.
http://wiki.edg.com/pub/Wg21cologne2019/CoreIssuesProcessingTeleconference2019-03-25/cwg_active.html#2407

[herb] Herb Sutter. 2019. Overload resolution changes as a result of P1185R2.
http://lists.isocpp.org/ext/2019/03/8704.php

[P0515R3] Herb Sutter, Jens Maurer, Walter E. Brown. 2017. Consistent comparison.
https://wg21.link/p0515r3

[P0732R2] Jeff Snyder, Louis Dionne. 2018. Class Types in Non-Type Template Parameters.
https://wg21.link/p0732r2

[P0848R2] Casey Carter and Barry Revzin. 2019. Conditionally Trivial Special Member Functions.
https://wg21.link/p0848r2

[P0905R1] Tomasz Kamiński, Herb Sutter, Richard Smith. 2018. Symmetry for spaceship.
https://wg21.link/p0905r1

[P1185R2] Barry Revzin. 2019. `<=>` `!=` `==`.
https://wg21.link/p1185r2

[P1186R2] Barry Revzin. 2019. When do you actually use `<=>`?
https://wg21.link/p1186r2

[P1190R0] David Stone. 2018. I did not order this! Why is it on my bill?
https://wg21.link/p1190r0

[P1614R1] Barry Revzin. 2019. The Mothership Has Landed: Adding `<=>` to the Library.
https://wg21.link/p1614r1

[smith] Richard Smith. 2019. Processing relational/spaceship operator rewrites.
http://lists.isocpp.org/core/2019/05/6420.php

[vdv.defaulted] Daveed Vandevoorde. 2019. Wording for deleted defaulted `operator!=`.
http://lists.isocpp.org/core/2019/05/6478.php

[vdv.reference] Daveed Vandevoorde. 2019. Generating comparison operators for classes with reference or anonymous union members.
http://lists.isocpp.org/core/2019/05/6462.php

[vdv.well-formed] Daveed Vandevoorde. 2019. Processing relational/spaceship operator rewrites.
http://lists.isocpp.org/core/2019/05/6419.php